Graph as an Abstract Data Type (ADT)

As discussed earlier in the course, an abstract data type (ADT) is a theoretical concept that defines a set of operations and their behavior without specifying the internal representation of the data or the algorithms used to implement those operations. It provides a high-level description of the data and the functions that can be performed on it.

Here are some of the operations can be performed on graphs:

1. Adding a new vertex
2. Removing a vertex
3. Adding an edge between two vertices
4. Removing an edge between two vertices
5. Getting a list of all the vertices
6. Checking if two graph nodes are adjacent to each other or not
7. Getting count of the total vertices in the graph
8. Getting count of the total edges in the graph
9. Getting a list of the graph edges
10. Getting neighbors of a graph vertex

There are a few other ADT operations involving graph searching and traversal. Searching and traversal operations require detailed illustrations. Thereby, we will discuss those in the Graph Traversals section.

Java

```java
import java.util.ArrayList;
import java.util.HashMap;

class Graph {
    private HashMap<Integer, ArrayList<Integer>> adjacencyList;

    public Graph() {
        adjacencyList = new HashMap<>();
    }

    public void addVertex(int vertex);
    // Add a new vertex to the graph.

    public void removeVertex(int vertex);
    // Remove a vertex from the graph.

    public void addEdge(int vertex1, int vertex2);
    // Add an edge between two vertices.

    public void removeEdge(int vertex1, int vertex2);
    // Remove an edge between two vertices.
```

Implementing the ADT operations

In the beginning of this section, we mentioned ten major and the most common ADT operations associated with the graphs. Let's discuss each operation individually - understanding how is it implemented.

1. add_vertex(vertex)

The function `add_vertex(vertex)` is a common operation in graph theory, where it adds a new vertex (also called a node or point) to a graph data structure. The vertex is a fundamental unit of a graph and represents an entity or an element. Adding a vertex expands the graph and creates potential connections (edges) between this new vertex and existing vertices.

In an adjacency list, each vertex is associated with a list of its adjacent vertices. We are using a dictionary or map for implementing the adjacency list. Thereby, we are able to store node key and value (s) associated with it.

For this course, we assume that graphs don't have any extra satellite data or values associated with it. Therefore, we will add an empty vector or list as a value attribute for the new graph node or vertex.

You can add a new vertex following the two main steps:

- Check if vertex already exists
    - Use predefined map or dictionary search methods
- If not, add it by inserting new key-value pair
    - Keep value of the new inserted vertex as empty list

Here is a generalized pseudocode:

```
Graph is represented as:

adjacencyList (map/dictionary):
  key: vertex
  value: list of adjacent vertices

Function addVertex(vertex)

  if vertex does not exist in adjacencyList

    add new key-value pair:
      key: vertex
      value: empty list
    end if

end Function
```

Let's now see the actual implementation of the `add_vertex()` method.

Java

```java
public void addVertex(int vertex) {
        if (!adjacencyList.containsKey(vertex)) {
            adjacencyList.put(vertex, new ArrayList<>());
        }
    }

```

2. add_edge(vertex1, vertex2)

Pseudocode

```
Function addEdge(vertex1, vertex2):

  insert vertex2 into adjacencyList[vertex1]
  insert vertex1 into adjacencyList[vertex2]

end Function
```

To implement the `add_edge()` function, we need to add an edge between `vertex1` and `vertex2` in the graph. We are using the `insert` method provided by the `std::unordered_map`. This method allows us to insert elements into the vector of neighbors directly using an iterator.

Here is the implementation:

Java

```java
public void addEdge(int vertex1, int vertex2) {
        adjacencyList.computeIfAbsent(vertex1, k -> new ArrayList<>()).add(vertex2);
        adjacencyList.computeIfAbsent(vertex2, k -> new ArrayList<>()).add(vertex1);
    }

```

A complete executable example

Java

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

class Pair<T, U> {
    private T first;
    private U second;

    public Pair(T first, U second) {
        this.first = first;
        this.second = second;
    }

    public T getFirst() {
        return first;
    }

    public U getSecond() {
        return second;
    }
```

Run Code Result | Stdout

You must run your code first

Execute

← Previous
Graph Representations

Next →
Graph Traversal - Depth First Search(DFS)

☐ Mark as Completed